

DIE EIGENE KI

LOKALE LLMS UND RAG FÜR KI-LÖSUNGEN

HALLO, ICH BIN DER BASTI 🖐️



- Sebastian Springer
- React & Node.js
- München

AGENDA

- LLMs lokal ausführen
- LangChain
- LangGraph
- RAG Grundlagen
- RAG Vorbereitungen
- RAG Applikation

LLM ÜBERBLICK

LLMS

1. Tokenisierung: Der Text wird in kleinere Einheiten, sogenannte Tokens, zerlegt. Diese können Wörter, Wortteile oder Zeichenfolgen sein.
2. Embedding: Jedes Token wird in einen Vektor im kontinuierlichen Raum umgewandelt, wobei semantisch ähnliche Tokens ähnliche Vektoren erhalten.
3. Vorhersage des nächsten Tokens: Das Modell berechnet die Wahrscheinlichkeit für jedes mögliche nächste Token basierend auf dem bisherigen Kontext.
4. Dekodierung: Anhand der berechneten Wahrscheinlichkeiten wird entschieden, welches Token als nächstes generiert wird, um den Text zu erzeugen.

RESSOURCEN

- CPU: für die Ausführung des Modells
- GPU: für die Ausführung des Modells
- RAM: vorhalten des Modells
- Größe: Llama 3.2 3B => Size 2GB => Memory 4GB

WELCHE LLMS?

- GPT
- Llama
- Qwen
- BERT
- T5
- Claude
- Mistral
- Grok
- Gemini
- Phi
- Gemma

Weitere Informationen zu LLMs

EUROPÄISCHE LLMS

- EuroLLM
- OpenGPT-X
- Leaderboard

OLLAMA

INSTALLATION

- Auf allen gängigen Plattformen als Installer/Paket verfügbar
- Als Docker-Container verfügbar
- Erreichbar über Kommandozeile oder <http://localhost:11434>

OLLAMA COMMANDS

- serve: Start ollama
- pull: Pull a model from a registry
- run: Run a model, pull if necessary
- list: List models
- show: Show information for a model
- ps: List running models
- stop: Stop a running model
- rm: Remove a model
- create: Create a model from a Modelfile
- cp: Copy a model
- push: Push a model to a registry
- help: Help about any command

OPEN WEBUI

- <https://openwebui.com/>
- Grafische Oberfläche für LLM-Runtimes
- Vergleichbar mit der ChatGPT UI
- Einfache Installation per Container
- Progressive Web App

OPEN WEBUI

```
docker run -d  
--network=host  
-v open-webui:/app/backend/data  
-e OLLAMA_BASE_URL=http://127.0.0.1:11434  
--name open-webui  
--restart always  
ghcr.io/open-webui/open-webui:main
```

OLLAMA API

- HTTP-API zur Verwaltung von Ollama
- POST /api/generate: Einmalige Anfrage mit einer direkten Antwort
- POST /api/chat: Anfragen mit Nachrichtenverlauf mit einer kontextbezogenen Antwort
- Weitere Endpunkte zum Hinzufügen von Modellen etc.
- [Ollama API](#)

OLLAMA.JS

- Komfortable Schnittstelle die die Kommunikation vereinfacht
- Unabhängig vom verwendeten Modell
- Streaming-Support
- Unterstützt neben Generation und Chat auch alle übrigen Funktionen
- Installation: `npm install ollama`
- [Doku](#)

LANGCHAIN

LANGCHAIN

- Open Source Framework für LLM-Apps
- Werkzeuge und Abstraktionen zur Integration von LLMs und Datenquellen
- Modularer Aufbau einer Applikation
- Verkettung von Modulen zu Applikationen
- Verfügbar in Python und JavaScript
- [Webseite](#)
- Installation: `npm install @langchain/core @langchain/ollama`

BESTANDTEILE VON LANGCHAIN

- Models: einheitliche Schnittstelle zu LLMs
- Messages: Kommunikationseinheiten für Input und Output
- Prompt-Templates: Vorlagen, in die Eingaben eingesetzt werden können
- History/Memory: Das Gedächtnis einer LLM-Applikation
- Tools: Funktionen, die vom LLM aufgerufen werden können
- Output Parser: Umwandlung der Ausgabe des Modells
- LCEL (LangChain Expression Language): Syntax zur Erstellung von Ketten

LANGCHAIN MIT TOOLS

TOOLS

Problemstellung: Modelle haben keine aktuellen Informationen und können nicht auf geschützte Daten zugreifen

- Erweiterung des LLMs: Tools ermöglichen es dem Modell, externe Funktionen auszuführen, z. B. API-Aufrufe, Berechnungen oder Datenbankabfragen.
- Definierte Schnittstelle: Jedes Tool hat einen Namen, eine Beschreibung und ein Schema für die erwarteten Eingaben und Ausgaben.
- Tool-Calling durch das LLM: Bei unterstützten Modellen kann das LLM selbstständig entscheiden, wann es ein Tool aufruft.
- Verschiedene Anwendungsfälle: Typische Tools sind Wetter-APIs, Suchmaschinen, Datenbanken, mathematische Berechnungen oder Dateiverarbeitung.

LANGGRAPH

LANGGRAPH

- Framework für stateful LLM-Applikationen
- Graphenstruktur zur Modellierung von Applikationen
- Verzweigungen und Schleifens

LANGGRAPH

- State: Globaler Zustand der Applikation
- Node: Funktionsblöcke der Applikation
- Edge: Verbindung zwischen den Nodes

LANGGRAPH BEISPIEL

EINFÜHRUNG IN RAG

EINFÜHRUNG IN RAG

- Kombination aus Retrieval (Informationsabruf) und Generation (Textgenerierung)
- Aktuelle Antworten und kontextbezogenes Wissen
- Pipeline mit Vektorsuche
- Verschiedene Verbesserungsmöglichkeiten entlang der Verarbeitungskette
- Wichtiges Element: Vorbereitung der Daten

AUFBAU EINER RAG-APPLIKATION

1. Vorbereitung

- 1. Datenquellen

- 2. Chunking

- 3. Embeddings

- 4. Vektorspeicher

2. Retrieval & Generation

- 1. Retrieval

- 2. Generation

DATENQUELLEN

DATENQUELLEN

- Unstrukturierte Daten: PDFs, Markdown, Webseiten
- Strukturierte Daten: APIs, Tabellen
- Vordefinierte Loader für z.B. PDF, Doc, etc
- Eigene Loader

TEXT SPLITTING

TEXT SPLITTING

- Längere Texte zerlegen für effizientere Verarbeitung (Kontextgröße)
- Verschiedene Strategien: Zeichenlänge, Wörter, Sätze, Absätze, semantische Einheiten
- Overlap: Überlappung, um den Kontext zu behalten

ZEICHENBASIERTES CHUNKING

- Text wird nach einer bestimmten Zeichenanzahl gesplittet (mit optionalem Overlap).
- Vorteile
 - Schnell und einfach
 - Sprachunabhängig
- Nachteile
 - Kann wörter und Sätze trennen
 - Keine semantische Aufteilung

WORTBASIERTES CHUNKING

- Splitting erfolgt nach einer festen Anzahl von Wörtern statt Zeichen.
- Kann mit regulären Ausdrücken (`\s+`) oder NLP-Bibliotheken wie `nltk` oder `spacy` umgesetzt werden.
- Vorteile
 - Trennt Texte nicht mitten in Wörtern
 - Funktioniert gut für natürliche Sprachen
- Nachteile
 - Begrenzte Kontrolle über Chunk-Länge
 - Kann Sätze trennen, wenn nicht zusätzlich darauf geachtet wird

SATZBASIERTES CHUNKING

- Der Text wird anhand von Satzzeichen (. ! ?) in Sätze segmentiert. Sätze werden so gruppiert, dass sie die maximale Chunk-Länge nicht überschreiten.
- Vorteile
 - Bewahrt semantische Einheit von Sätzen
 - Gut für Embeddings, NLP und Textzusammenfassungen
- Nachteile
 - Kann zu unterschiedlich großen Chunks führen
 - Mehr Verarbeitungsschritte erforderlich

ABSATZBASIERTES CHUNKING

- Trennung erfolgt anhand von doppelten Zeilenumbrüchen oder bestimmten Abschnittsmarkern.
- Vorteile:
 - Bewahrt logische Textstruktur (z. B. Kapitel, Absätze)
 - Ideal für große Dokumente mit klarer Gliederung
- Nachteile:
 - Kann sehr große Chunks erzeugen, wenn Absätze lang sind
 - Nicht optimal für gleichmäßige Chunk-Größe

SEMANTISCHES CHUNKING

- Nutzt die Struktur des Dokuments, z. B. Markdown-Überschriften, HTML-Tags oder JSON-Knoten.
- Trennung erfolgt nach logischen Einheiten, z. B. H2-Überschriften in Markdown oder HTML section Tags.
- Vorteile:
 - Perfekt für Dokumente mit klaren Hierarchien (z. B. wissenschaftliche Artikel, Blogs)
 - Bessere semantische Gruppierung als Zeichen- oder Wortbasiertes Splitting
- Nachteile:
 - Aufwändiger zu implementieren
 - Je nach Format unterschiedliche Parsing-Techniken nötig

EMBEDDINGS

EMBEDDINGS

- Numerische Vektordarstellungen von Texten, die ihre semantische Bedeutung erfassen
- Ähnlichkeiten können berechnet werden. Vektor von Katze und Hund liegen näher zusammen als der von Auto
- Vektorsuche statt Keyword-Suche - Semantisch ähnliche Inhalte werden gefunden
- Einsatz von hochdimensionalen Vektoren

EMBEDDINGS

Dimensionen	Speicherbedarf	Suchgenauigkeit	Qualität
< 256	Sehr gering	Niedrig	Schlecht
384 - 512	Mittel	Gut	Schlecht
768 - 1024	Hoch	Sehr gut	Leistungsfähig
> 1536	Sehr hoch	Geringer Zusatznutzen	Leistungsfähig

EMBEDDINGS

- nomic-embed → 768 oder 1024
- OpenAI text-embedding-ada-002 → 1536
- sentence-transformers (BERT & Co.) → 384, 512, 768, 1024
- FAISS PCA-Reduktion → Frei wählbar

VECTOR STORAGE

VECTOR STORAGE

- Vektoren zur semantischen Suche durch Abstandsberechnung zweier Vektoren
- Vorgehen:
 1. Daten in Embeddings umwandeln
 2. Speichern der Vektoren
 3. Ähnlichkeitssuche z.B. mit k-NN oder ANN

VECTOR STORAGE

Vektordatenbank	Stärken	Sprache	Skalierbarkeit
Milvus	Beste Wahl für RAG & AI	Python, Go	Hoch skalierbar (Cloud & On-Prem)
FAISS	Schnell & effizient	Python, C++	Optimal für große Embedding-Sets
Weaviate	Vektor-Datenbank mit integrierter KI	Python, REST API	Sehr flexibel
Pinecone	Cloud-native, keine eigene Infrastruktur nötig	Python, REST API	Skalierbar, aber kostenpflichtig
Qdrant	Rust-basiert, Open-Source	Python, Rust	Optimiert für schnelle Suchen

LANGCHAIN MEMORY VECTOR STORE

- Speichert Embeddings direkt im Arbeitsspeicher
- Schnelle Ähnlichkeitssuche, gut für kurzlebige Sessions
- Einfache Integration
- Ideal für Entwicklungs- und Testzwecke

MILVUS

- Skalierbare, leistungsfähige Open-Source-Vektordatenbank für Ähnlichkeitssuche, KI und ML
- Optimiert auf Vektorsuche mit großer Menge an hochdimensionalen Embeddings
- Unterstützt mehrere **Indexierungs-Algorithmen**

MILVUS

- etcd: Verteilte Key-Value-Datenbank, die Milvus zum speichern von Metadaten verwendet
- minio: Performanter S3-kompatibler Objektspeicher zur Speicherung von Vektordaten und Indexdateien
- Milvus: Die Vektordatenbank selbst
- Attu: Open-Source-UI für Milvus für die visuelle Verwaltung der Datenbank

MILVUS

- Zugriff über [JavaScript SDK](#)
- Verwaltung von Collections und Partitions
 - Insert, Update und Delete von Einträgen
- Single-Vector Suche, Hybrid Suche (mehrere ANN Suchen parallel, rerank für bessere Ergebnisse)

WRAP UP - VORBEREITUNG

WRAP UP -VORBEREITUNG

1. Datenquelle
2. Chunking
3. Embeddings
4. Vector Storage

PHASE 2: RAG

RAG

1. Retrieval
2. Generation

RETRIEVAL

RETRIEVAL

- Ähnlichkeitssuche in der Vektordatenbank, um Input für den Context zu erzeugen
- Vorgehensweise:
 1. Suchtext in Vektor umwandeln
 2. Ähnlichkeitssuche durchführen
 3. Dokumente zurückgeben

RETRIEVAL

- Gleiche Embeddings verwenden
- Korrekten Collectionnamen nutzen
- Anzahl der Ergebnisse

AUGMENTATION

AUGMENTATION

- Prompt-Template
- Model
- Output Parser

TYPISCHE USE CASES

TYPISCHE USE CASES

- Dokumentenbasierte Chatbots - RAG ermöglicht Chatbots den Zugriff auf große Wissensbasen, um präzise und aktuelle Antworten aus Dokumenten bereitzustellen.
- Technische Support-Assistenten - Nutzeranfragen werden mit technischen Handbüchern oder Support-Datenbanken abgeglichen, um kontextspezifische Lösungen zu liefern.
- Rechtliche Recherche - RAG kann juristische Texte durchsuchen und zusammenfassen, um Rechtsanwälten relevante Präzedenzfälle und Gesetze bereitzustellen.
- Medizinische Wissenssysteme - Ärzte oder Forscher können gezielt in medizinischen Studien und Leitlinien nach aktuellen Forschungsergebnissen suchen.
- Wissenschaftliche Literaturrecherche - Forscher erhalten kontextbezogene Zusammenfassungen und relevante Paper aus großen wissenschaftlichen Archiven.
- Personalisierte Empfehlungssysteme - RAG kann Nutzerhistorien mit aktuellen Informationen kombinieren, um individuell zugeschnittene Vorschläge zu generieren.
- Unternehmensinterne Wissensmanagement-Systeme - Mitarbeiter können komplexe interne Datenbanken durchsuchen und präzise Antworten auf spezifische Fragen erhalten.
- Automatisierte Zusammenfassung von Berichten - Große Mengen an Finanz-, Markt- oder Geschäftsberichten können durchsucht und prägnant zusammengefasst werden.

FALLSTRICKE

FALLSTRICKE

- Unzureichende Retrieval-Qualität
- Halluzinationen trotz externer Daten
- Veraltete oder irrelevante Dokumente
- Zu lange oder zu kurze Chunks
- Sprachunterstützung bei den Embeddings
- Fehlende Kontextualisierung durch das LLM
- Skalierungsprobleme bei großen Datenmengen
- Bias in den Trainings- oder Retrieval-Daten
- Kombination von mehreren Retrieval-Ergebnissen
- Fehlende Erklärbarkeit und Nachvollziehbarkeit
- Sicherheits- und Datenschutzrisiken

BEST PRACTICES

OPTIMIERUNG DER DATENQUELLE UND DES RETRIEVAL-MECHANISMUS

OPTIMIERUNG DER EMBEDDINGS

CHUNKING-STRATEGIE VERBESSERN

- Optimale Chunk-Größe wählen: Typischerweise 200-500 Token, um genug Kontext zu behalten, aber nicht zu viel irrelevante Information einzuschließen.
- Overlapping Chunks nutzen: 10-20% Overlap zwischen Chunks, um den Zusammenhang zwischen Abschnitten beizubehalten.
- Dynamisches Chunking: Falls möglich, Absätze oder semantisch zusammenhängende Blöcke statt fixer Token-Längen verwenden.

RETRIEVAL-QUALITÄT MAXIMIEREN

- Mehrere relevante Dokumente abrufen: Statt nur eines Dokuments Top-k (z. B. 3-5) relevante Treffer nutzen.
- Re-Ranking anwenden: Durch Cross-Encoders oder BM25-Re-Ranking die relevantesten Treffer priorisieren.
- Kontextgewichtung nutzen: Falls mehrere Dokumente geliefert werden, wichtigere Abschnitte stärker gewichten.

GENERATIONSMODELL FEINABSTIMMEN

- Prompt Engineering für bessere Antworten:
- Explizit angeben, dass das Modell nur aus den abgerufenen Dokumenten antworten darf.
- Falls keine relevanten Treffer existieren, eine Fallback-Antwort (z. B. „Keine verlässlichen Informationen gefunden“) generieren lassen.
- Few-Shot Learning für spezifische Anwendungsfälle: Falls möglich, mit Beispielen arbeiten, um das Modell auf die richtige Antwortstruktur zu lenken.
- Zusätzliche Guardrails für Halluzinationen: Falls keine passenden Retrieval-Ergebnisse gefunden werden, die Generierung deaktivieren.

MEHRSTUFIGE RAG-ARCHITEKTUREN NUTZEN

- Hierarchische Retrieval-Strategien: Erst in groben Kategorien suchen, dann feingranulare Dokumente abrufen.
- Query Expansion nutzen: Falls nötig, die Nutzeranfrage um Synonyme oder reformulierte Versionen ergänzen.
- Feedback-Mechanismen einbauen: Nutzerfeedback für falsch oder richtig beantwortete Anfragen sammeln und für Verbesserungen nutzen.

SKALIERBARKEIT UND PERFORMANCE OPTIMIEREN

- Vektorindex regelmäßig aktualisieren: Veraltete oder falsche Embeddings können die Retrieval-Qualität verschlechtern.
- Caching für häufige Anfragen: Wiederkehrende Suchanfragen zwischenspeichern, um Latenz zu reduzieren.
- Paralleles Processing nutzen: Falls viele Dokumente analysiert werden müssen, mehrere Retrieval- und LLM-Abfragen parallelisieren.

TRANSPARENZ UND DEBUGGING ERMÖGLICHEN

- Quellenverweise anzeigen: Falls das Modell eine Antwort generiert, immer eine Referenz auf die verwendeten Dokumente geben.
- Score-basierte Antwortvalidierung: Falls die Retrieval-Ergebnisse nur eine niedrige Relevanz haben, eine Unsicherheitswarnung oder keine Antwort ausgeben.
- Logging und Evaluation implementieren: Metriken wie Recall, MRR (Mean Reciprocal Rank) und NDCG (Normalized Discounted Cumulative Gain) nutzen.